

ELEC 5803:

Behavioral Synthesis of ICs

Simple design examples

Note: These slides might have been updated, please check.

Credits:

1. Arash Ahmadi, Electronic Department, Carleton University, 2025.

Outline

- Introduction
- Custom Single-Purpose Processors
- A brief review on logic circuit design
- Design Examples
- More Design Challenges

Custom Single-Purpose Processors

RT-level custom single-purpose processor design

Conception de processeur personnalisé à usage unique
de niveau RT

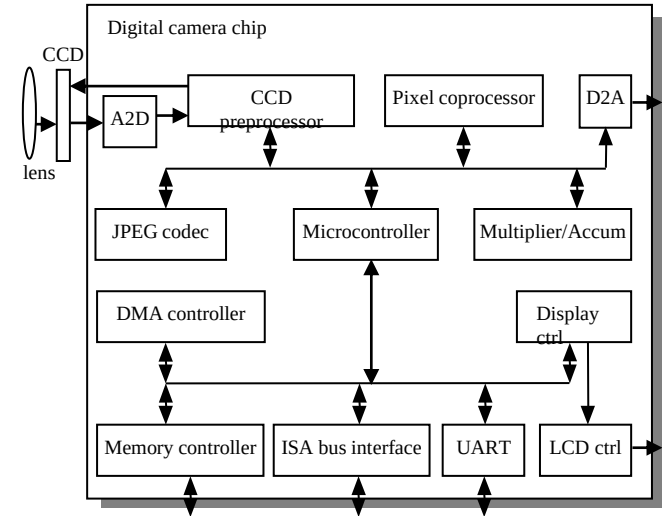
Introduction

→ Processor

- Digital circuit that performs a computation tasks
- Controller and datapath
- General-purpose: variety of computation tasks
- Single-purpose: one particular computation task
- Custom single-purpose: non-standard task

→ A custom single-purpose processor may be

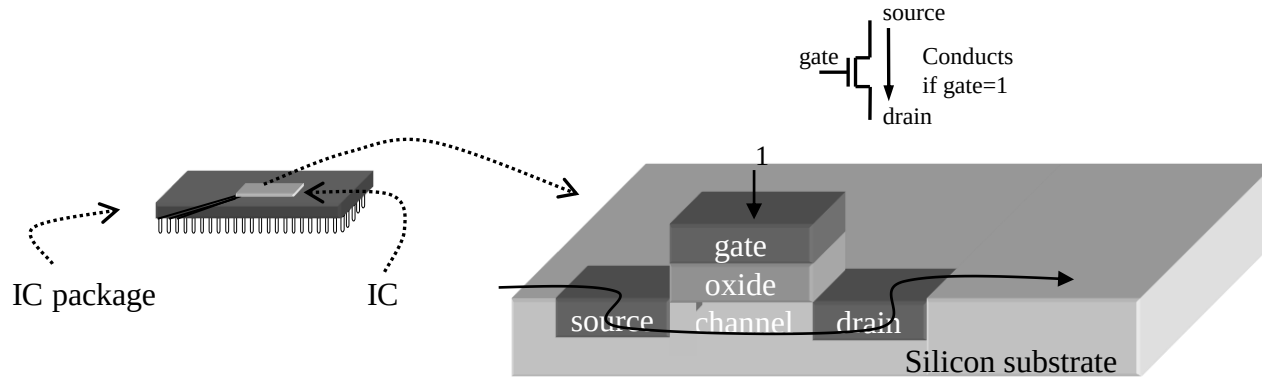
- Fast, small, low power
- But, high NRE costs, longer time-to-market, less flexible



CMOS transistor on silicon

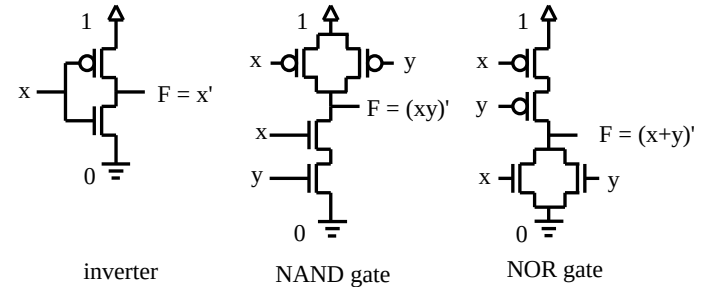
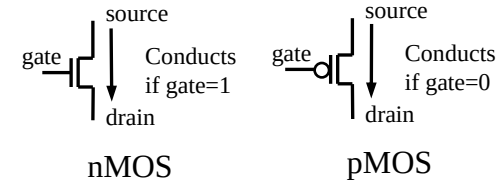
→ Transistor

- The basic electrical component in digital systems
- Acts as an on/off switch
- Voltage at “gate” controls whether current flows from source to drain
- Don’t confuse this “gate” with a logic gate



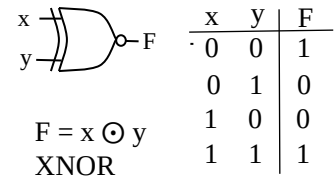
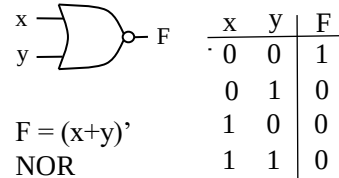
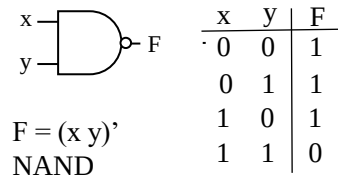
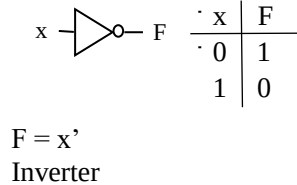
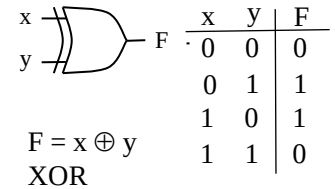
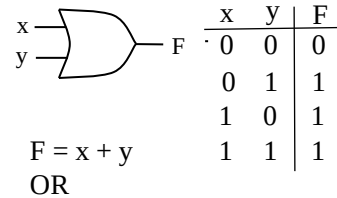
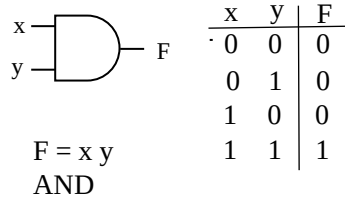
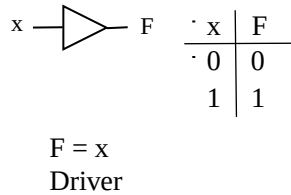
CMOS transistor implementations

- Complementary Metal Oxide Semiconductor
- We refer to logic levels
 - Typically 0 is 0V, 1 is 5V
- Two basic CMOS types
 - nMOS conducts if gate=1
 - pMOS conducts if gate=0
 - Hence “complementary”
- Basic gates
 - Inverter, NAND, NOR



Basic logic gates

→ Logic gates



Combinational logic design

→ Combinational logic design for small circuits.

A) Problem description

y is 1 if a is to 1, or b and c are 1. z is 1 if b or c is to 1, but not both, or if all are 1.

D) Minimized output equations

y	a	bc			
		00	01	11	10
0		0	0	1	0
1		1	1	1	1

$$y = a + bc$$

z	a	bc			
		00	01	11	10
0		0	1	0	1
1		0	1	1	1

$$z = ab + b'c + bc'$$

B) Truth table

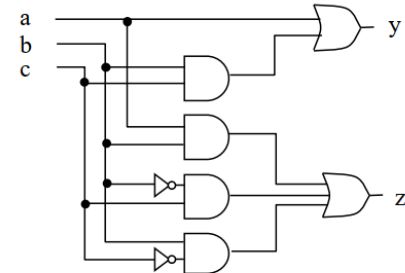
Inputs			Outputs	
a	b	c	y	z
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	1	0
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

C) Output equations

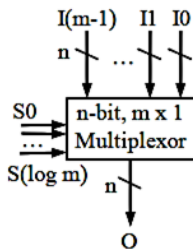
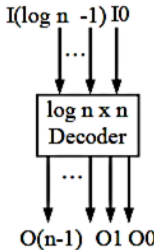
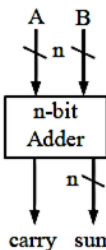
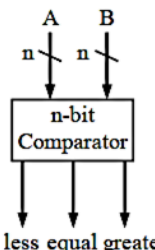
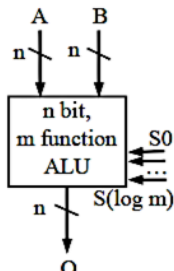
$$y = a'bc + ab'c' + ab'c + abc' + abc$$

$$z = a'b'c + a'bc' + ab'c + abc' + abc$$

E) Logic Gates

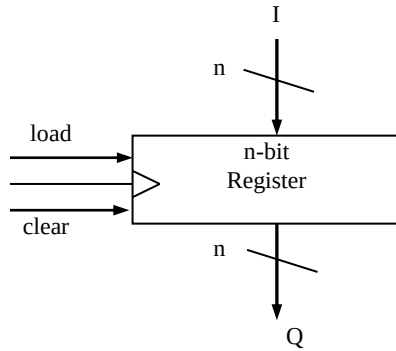


Combinational components

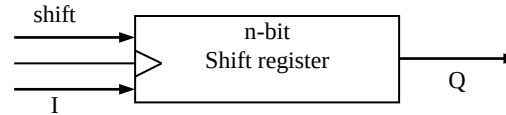
				
$O =$ I_0 if $S=0..00$ I_1 if $S=0..01$ $\dots$ $I_{(m-1)}$ if $S=1..11$	$O_0 = 1$ if $I=0..00$ $O_1 = 1$ if $I=0..01$ $\dots$ $O_{(n-1)} = 1$ if $I=1..11$	$sum = A + B$ (first n bits) $carry = (n+1)$'th bit of $A+B$	$less = 1$ if $A < B$ $equal = 1$ if $A = B$ $greater = 1$ if $A > B$	$O = A \text{ op } B$ op determined by S.
	With enable input $e \rightarrow$ all O's are 0 if $e=0$	With carry-in input $C_i \rightarrow$ $sum = A + B + C_i$		May have status outputs carry, zero, etc.

Sequential components

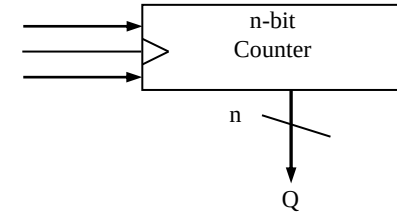
→ Sequential circuit building blocks



$Q =$
0 if clear=1,
I if load=1 and clock=1,
Q(previous) otherwise.



$Q = \text{lsb}$
- Content shifted
- I stored in msb



$Q =$
0 if clear=1,
 $Q(\text{prev})+1$ if count=1 and clock=1.

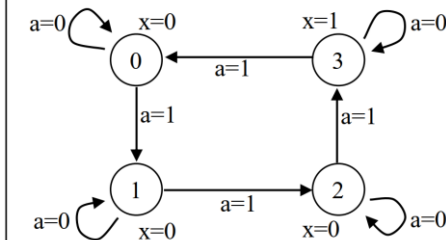
Sequential logic design

- Given this implementation model
 - Sequential logic design quickly reduces to combinational logic design

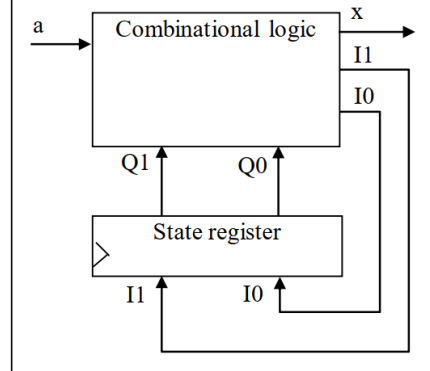
A) Problem Description

You want to construct a clock divider. Slow down your pre-existing clock so that you output a 1 for every four clock cycles

B) State Diagram



C) Implementation Model

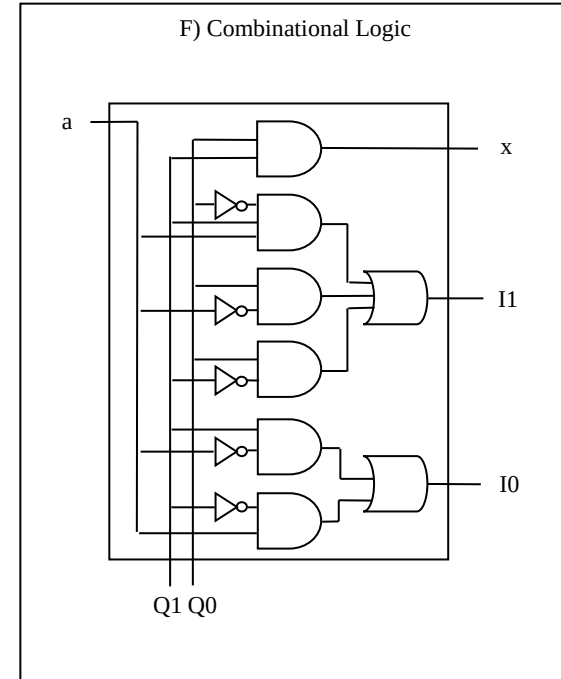
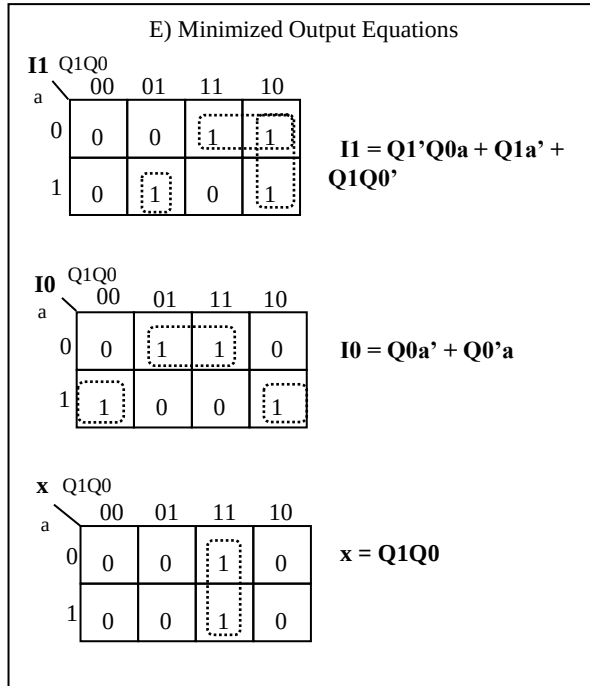


D) State Table (Moore-type)

Inputs			Outputs		
Q1	Q0	a	I1	I0	x
0	0	0	0	0	0
0	0	1	0	1	
0	1	0	0	1	0
0	1	1	1	0	
1	0	0	1	0	0
1	0	1	1	1	
1	1	0	1	1	1
1	1	1	0	0	

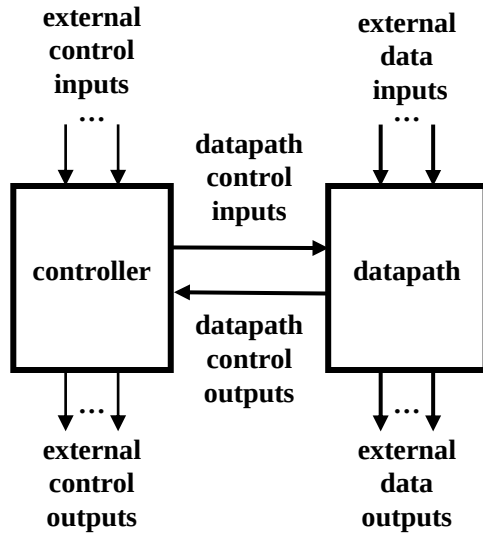
Sequential logic design (cont.)

→ Sequential circuit design



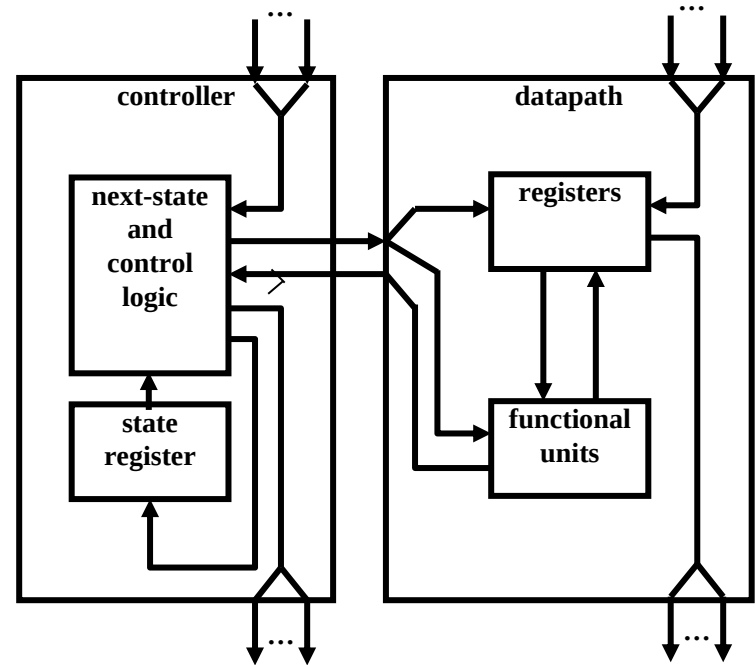
Custom single-purpose processor basic model

→ Datapath-Controller Design Approach



controller and datapath

>



a view inside the controller and datapath

Design Examples

Simple Step by Step Designs

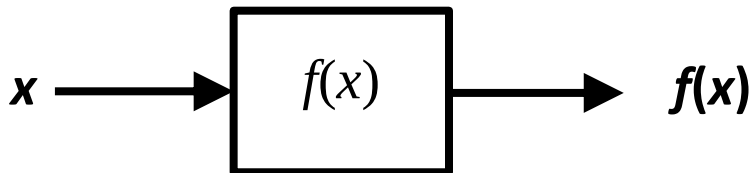
Conceptions simples étape par étape

Example 1: A first order polynomial

- We want to design a system which receives an input number (x) at each rising edge of the clock and produces an output (y), which is calculated as:

$$f(x) = ax + b$$

- where a and b are constants and x is a variable.

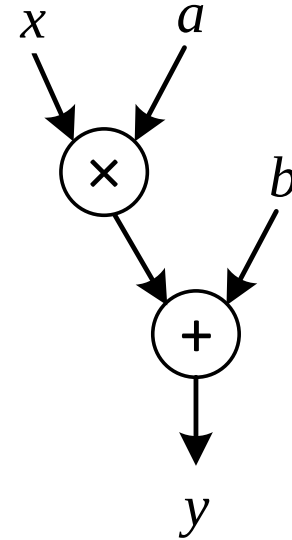


Example 1: A first order polynomial

Description analysis:

- What are the required functional units?
- In arithmetic units, what is the data format (floating point or fixed point)?
- What is the bit-width of the data?

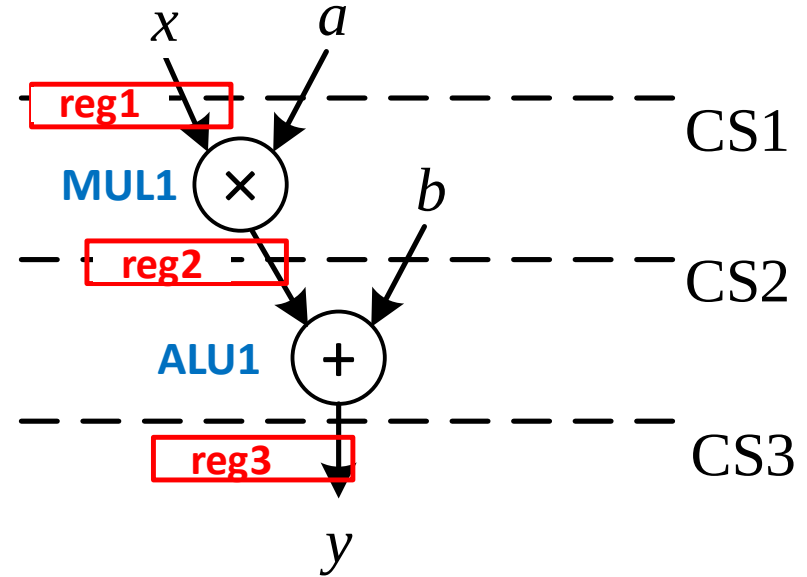
Operation	Functional Unit	Quantity
Multiplication	Multiplier	1
Addition	ALU	1



Example 1: A first order polynomial

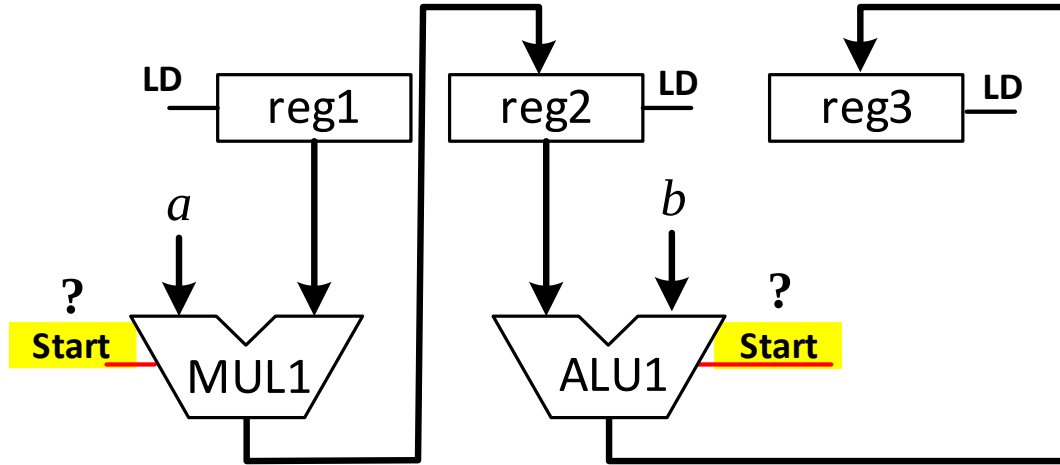
Datapath Design:

- Draw the dataflow Graph (DFG).
- Create the scheduling of the operations.
- Insert required registers.
- How long (clock steps) it take to execute?
- How many functional units (FU) is needed?
- Explore:
 - scheduling,
 - binding, and
 - allocation
- to find the optimum design.



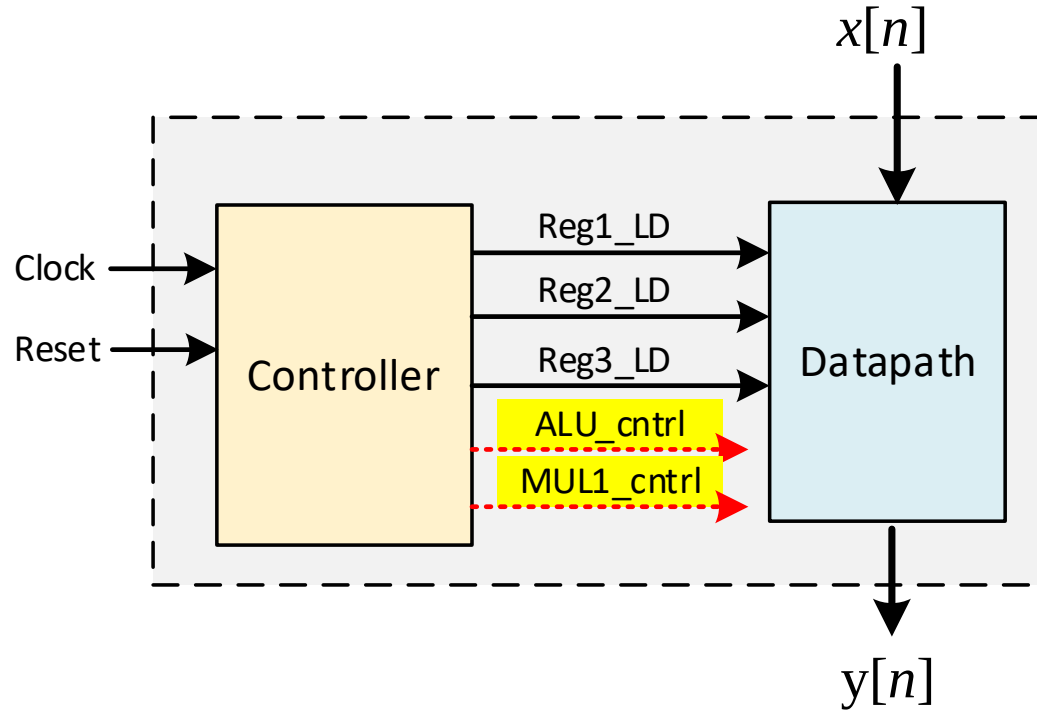
Example 1: A first order polynomial

→ Datapath hardware



Example 1: A first order polynomial

→ System view



Example 1: A first order polynomial

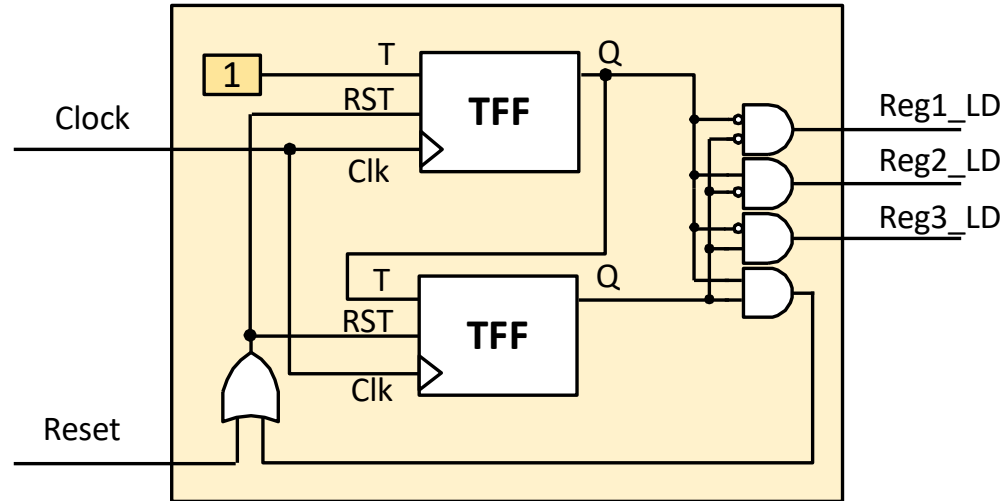
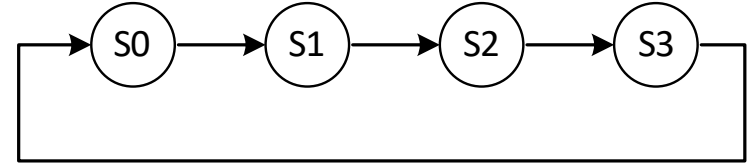
- Design the controller:
- Analyze execution sequence of the algorithm,
- Examine how many steps are required to execute the algorithm.
- Design the State Machine based operations sequence in the scheduling diagram. In this example:

State Codes	Current State	Operations / Functional Unit	Control Signals	Next State
00	S0	System Reset	RST	S1
01	S1	Load x to reg1	reg1_LD	S2
10	S2	MUL1, Load MUL1 to reg2	MUL_start, reg2_LD	S3
11	S3	ADD1, Load ADD1 to reg3	ADD_start, reg3_LD	S0



Example 1: A first order polynomial

→ Controller Hardware (complete the design)



Register Module: Verilog vs SystemVerilog

→ Notes: ``reg`` vs ``logic``, ``always`` @ vs ``always_ff`` @, ``0`` for cleaner reset

```
// Verilog Register
module reg_sync #(parameter WIDTH=16) (
    input clk, rst, ld,
    input [WIDTH-1:0] d,
    output reg [WIDTH-1:0] q
);
    always @(posedge clk or posedge rst) begin
        if (rst)
            q <= 0;
        else if (ld)
            q <= d;
    end
endmodule
```

```
// SystemVerilog Register
module reg_sync #(parameter WIDTH=16) (
    input logic clk, rst, ld,
    input logic [WIDTH-1:0] d,
    output logic [WIDTH-1:0] q
);
    always_ff @(posedge clk) begin
        if (rst)
            q <= '0;
        else if (ld)
            q <= d;
    end
endmodule
```

Functional Units: Verilog vs SystemVerilog

→ Notes: SystemVerilog uses ``logic`` consistently, no `reg/wire` confusion

```
// Verilog Multiplier
module multiplier #(parameter WIDTH=16)(
    input [WIDTH-1:0] a, b,
    output [2*WIDTH-1:0] y
);
    assign y = a * b;
endmodule

// Verilog Adder
module adder #(parameter WIDTH=16)(
    input [WIDTH-1:0] a, b,
    output [WIDTH-1:0] y
);
    assign y = a + b;
endmodule
```

```
// SystemVerilog Multiplier
module multiplier #(parameter WIDTH=16)(
    input logic [WIDTH-1:0] a, b,
    output logic [2*WIDTH-1:0] y
);
    assign y = a * b;
endmodule

// SystemVerilog Adder
module adder #(parameter WIDTH=16)(
    input logic [WIDTH-1:0] a, b,
    output logic [WIDTH-1:0] y
);
    assign y = a + b;
endmodule
```

Datapath for $y = ax + b$

→ Notes: Named port mapping in SV improves readability and maintainability

```
// Verilog Datapath
module datapath #(parameter WIDTH=16)(
    input clk, rst, ld_x, ld_mul, ld_add,
    input [WIDTH-1:0] x_in, a, b,
    output [WIDTH-1:0] y_out
);
    wire [WIDTH-1:0] x_reg, mul_reg, add_reg;
    wire [2*WIDTH-1:0] mul_out;
    wire [WIDTH-1:0] add_out;

    reg_sync #(WIDTH) r1 (clk,rst,ld_x,x_in,x_reg);
    reg_sync #(WIDTH) r2 (clk,rst,ld_mul,mul_out[WIDTH-1:0],mul_reg);
    reg_sync #(WIDTH) r3 (clk,rst,ld_add,add_out,add_reg);

    multiplier #(WIDTH) m1 (a, x_reg, mul_out);
    adder        #(WIDTH) a1 (mul_reg, b, add_out);

    assign y_out = add_reg;
endmodule
```


Datapath for $y = ax + b$

→ Notes: Named port mapping in SV improves readability and maintainability

```
// SystemVerilog Datapath
module datapath #(parameter WIDTH=16)(
    input logic clk, rst,
    input logic ld_x, ld_mul, ld_add,
    input logic [WIDTH-1:0] x_in, a, b,
    output logic [WIDTH-1:0] y_out
);
    logic [WIDTH-1:0] x_reg, mul_reg, add_reg;
    logic [2*WIDTH-1:0] mul_out;
    logic [WIDTH-1:0] add_out;

    reg_sync #(WIDTH) r1 (.clk(clk), .rst(rst), .ld(ld_x), .d(x_in), .q(x_reg));
    reg_sync #(WIDTH) r2 (.clk(clk), .rst(rst), .ld(ld_mul), .d(mul_out[WIDTH-1:0]), .q(mul_reg));
    reg_sync #(WIDTH) r3 (.clk(clk), .rst(rst), .ld(ld_add), .d(add_out), .q(add_reg));

    multiplier #(WIDTH) m1 (.a(a), .b(x_reg), .y(mul_out));
    adder          #(WIDTH) a1 (.a(mul_reg), .b(b), .y(add_out));

    assign y_out = add_reg;
endmodule
```

Controller FSM: Verilog vs SystemVerilog

→ **Notes:** SystemVerilog `enum` make state machines self-documenting and safer

```
// Verilog FSM
module controller(input clk, rst,
                  output reg ld_x, ld_mul, ld_add);
    reg [1:0] state, next_state;
    localparam S0=2'b00, S1=2'b01,
               S2=2'b10, S3=2'b11;

    always @(posedge clk or posedge rst)
        if (rst) state <= S0;
        else     state <= next_state;

    always @(*) begin
        case (state)
            S0: next_state=S1;
            S1: next_state=S2;
            S2: next_state=S3;
            S3: next_state=S0;
        endcase
    end

    always @(*) begin
        ld_x=0; ld_mul=0; ld_add=0;
        case (state)
            S1: ld_x=1;
            S2: ld_mul=1;
            S3: ld_add=1;
        endcase
    end
endmodule
```

Controller FSM: Verilog vs SystemVerilog

→ **Notes:** SystemVerilog **enum** make state machines self-documenting and safer

```
// SystemVerilog FSM
module controller(input logic clk, rst,
                  output logic ld_x, ld_mul, ld_add);

    typedef enum logic [1:0] {S0, S1, S2, S3} state_t;
    state_t state, next_state;

    always_ff @(posedge clk) begin
        if (rst) state <= S0;
        else     state <= next_state;
    end

    always_comb begin
        case (state)
            S0: next_state=S1;
            S1: next_state=S2;
            S2: next_state=S3;
            S3: next_state=S0;
        endcase
    end

    always_comb begin
        ld_x=0; ld_mul=0; ld_add=0;
        case (state)
            S1: ld_x=1;
            S2: ld_mul=1;
            S3: ld_add=1;
        endcase
    end
endmodule
```

Top-Level Integration

→ **Notes:** SV version uses consistent **logic** types and named connections

```
// Verilog Top-Level
module poly_system #(parameter WIDTH=16)(
    input clk, rst,
    input [WIDTH-1:0] x_in, a, b,
    output [WIDTH-1:0] y_out
);
    wire ld_x, ld_mul, ld_add;

    datapath #(WIDTH)
        DP(clk,rst,ld_x,ld_mul,ld_add,x_in,a,b,y_out);
    controller CTRL(clk,rst,ld_x,ld_mul,ld_add);
endmodule
```

```
// SystemVerilog Top-Level
module poly_system #(parameter WIDTH=16)(
    input logic clk, rst,
    input logic [WIDTH-1:0] x_in, a, b,
    output logic [WIDTH-1:0] y_out
);
    logic ld_x, ld_mul, ld_add;

    datapath #(WIDTH) DP(.clk(clk),.rst(rst),
        .ld_x(ld_x),.ld_mul(ld_mul),.ld_add(ld_add),
        .x_in(x_in),.a(a),.b(b),.y_out(y_out));
    controller CTRL(.clk(clk),.rst(rst),
        .ld_x(ld_x),.ld_mul(ld_mul),.ld_add(ld_add));
endmodule
```

Testbench: Verilog vs SystemVerilog

→ Notes: SystemVerilog simplifies testbenches with unified **logic** types

```
// Verilog Testbench
module tb_poly;
    reg clk=0, rst=1;
    reg [15:0] x_in, a, b;
    wire [15:0] y_out;

    poly_system dut(clk,rst,x_in,a,b,y_out);

    always #5 clk = ~clk;

    initial begin
        #10 rst=0;
        a=2; b=3;
        x_in=5; #50;
        x_in=7; #50;
        $finish;
    end

    initial
        $monitor("t=%0t, x=%d,
y=%d", $time, x_in, y_out);
endmodule
```

```
// SystemVerilog Testbench
module tb_poly;
    logic clk=0, rst=1;
    logic [15:0] x_in, a, b, y_out;

    poly_system dut(.clk(clk),.rst(rst),.x_in(x_in),
        .a(a),.b(b),.y_out(y_out));

    always #5 clk = ~clk;

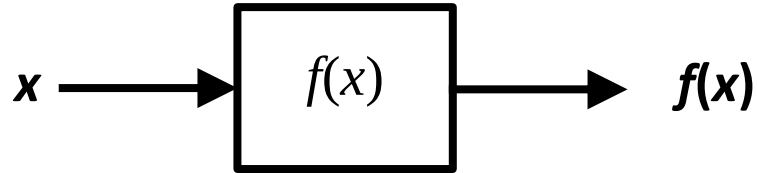
    initial begin
        #10 rst=0;
        a=2; b=3;
        x_in=5; #50;
        x_in=7; #50;
        $finish;
    end

    initial
        $monitor("t=%0t, x=%d, y=%d", $time, x_in, y_out);
endmodule
```

Example 2: A second order polynomial

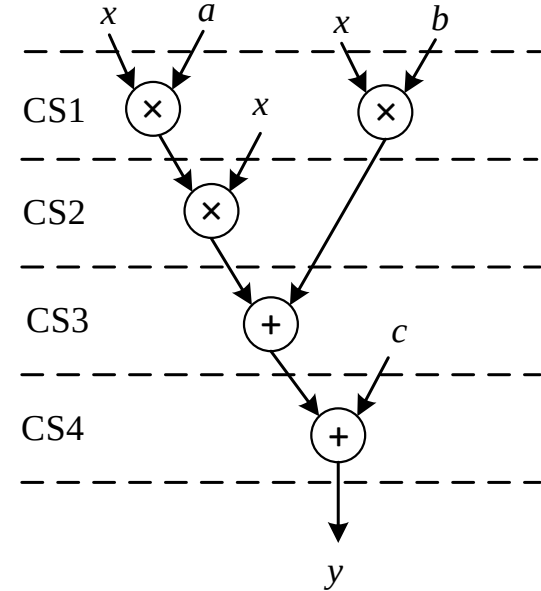
→ $f(x) = ax^2 + bx + c$

→ a , b and c are constants and x is a variable.



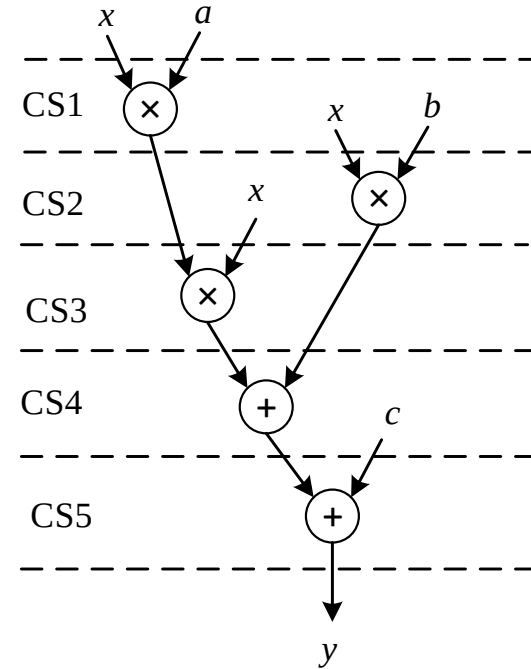
Example 2: A second order polynomial

- Scheduling option 1
- Minimum required functional units per control cycle:
 - 2 × MUL
 - 1 × ALU
- Number of control (clock) cycles:
 - 4



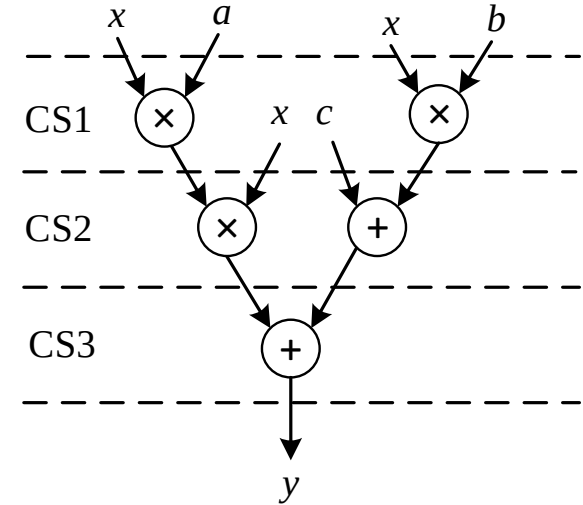
Example 2: A second order polynomial

- Scheduling option 2
- Minimum required functional units per control cycle:
 - 1 × MUL
 - 1 × ALU
- Number of control (clock) cycles:
 - 5



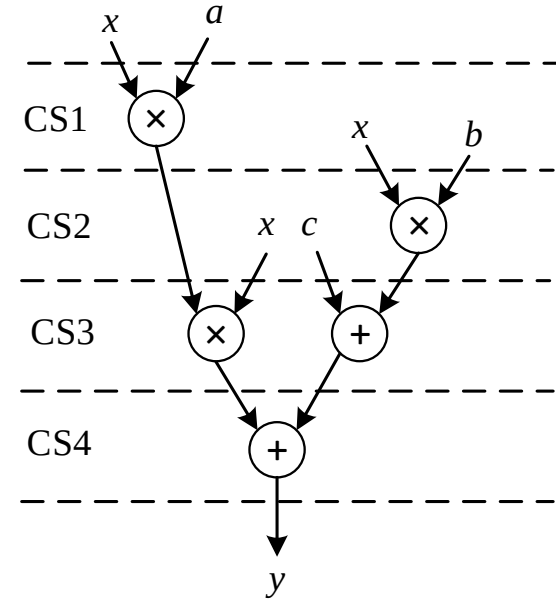
Example 2: A second order polynomial

- Scheduling option 3
- Minimum required functional unites per control cycle:
 - 2 × MUL
 - 1 × ALU
- Number of control (clock) cycles:
 - 3



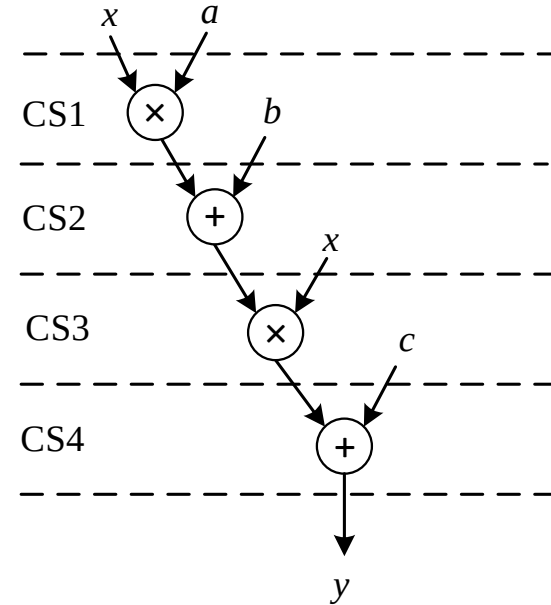
Example 2: A second order polynomial

- Scheduling option 4
- Minimum required functional unites per control cycle:
 - 1 × MUL
 - 1 × ALU
- Number of control (clock) cycles:
 - 4



Example 2: A second order polynomial

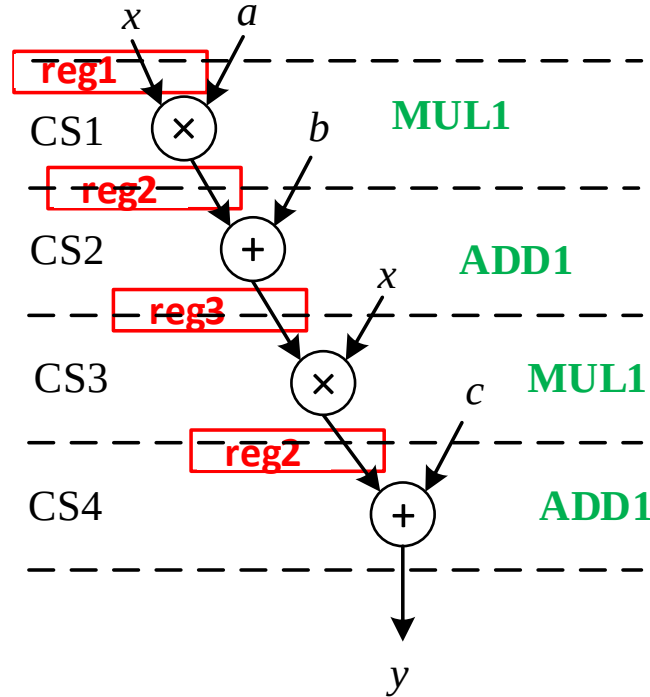
- Scheduling option 5
- Minimum required functional unites per control cycle:
 - 1 × MUL
 - 1 × ALU
- Number of control (clock) cycles:
 - 4



Example 2: A second order polynomial

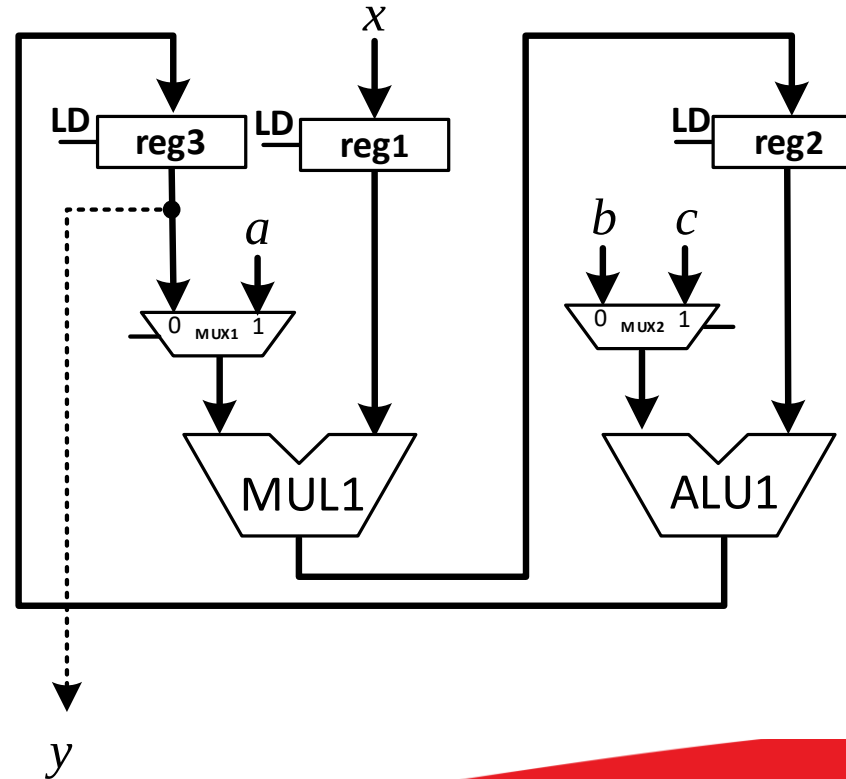
→ For Scheduling 5:

- Binding and Allocation
- Registration Insertion



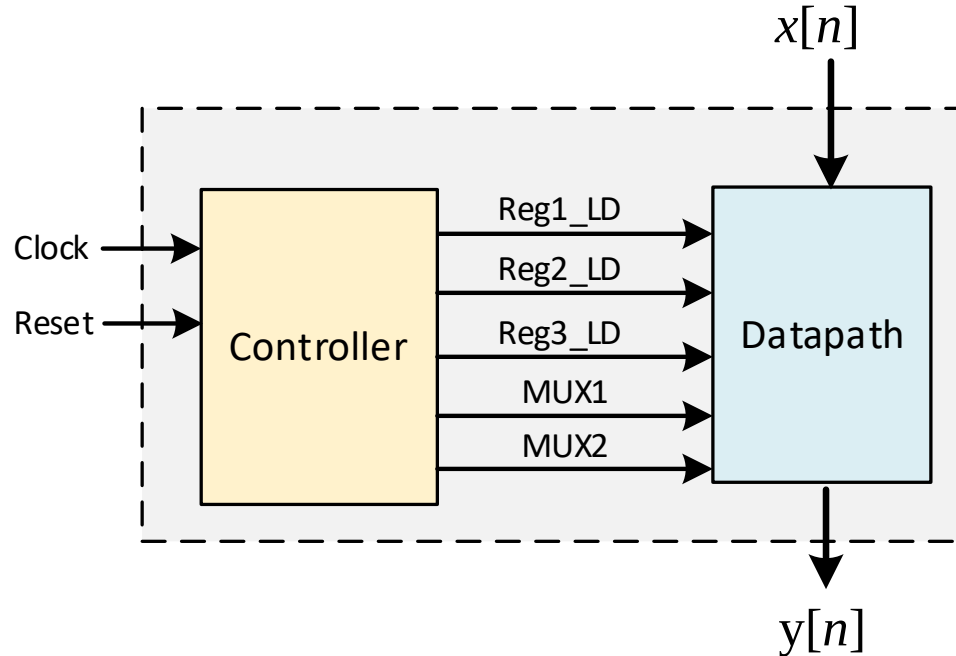
Example 2: A second order polynomial

→ Datapath design



Example 2: A second order polynomial

→ Controller Design: Control signal identification



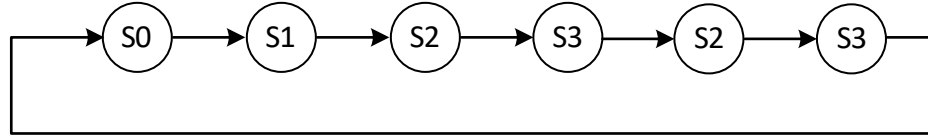
Example 2: A second order polynomial

→ Controller Design: Control signal sequence

State Codes	Current State	Operations / Functional Unit	Control Signals	Next State
000	S0	System Reset	RST=1	S1
001	S1	$x \rightarrow \text{reg1},$	reg1_LD=1,	S2
010	S2	$(a * \text{reg1}) \rightarrow \text{reg2}$	MUX1=1, reg2_LD=1	S3
011	S3	$(\text{reg2} + b) \rightarrow \text{reg3}$	MUX2=0, reg3_LD=1	S4
100	S4	$(x * \text{reg3}) \rightarrow \text{reg2}$	MUX1=0, reg2_LD=1	S5
101	S5	$(\text{reg2} + c) \rightarrow \text{reg3}$	MUX2=1, reg3_LD=1	S0

Example 2: A second order polynomial

→ FSM design



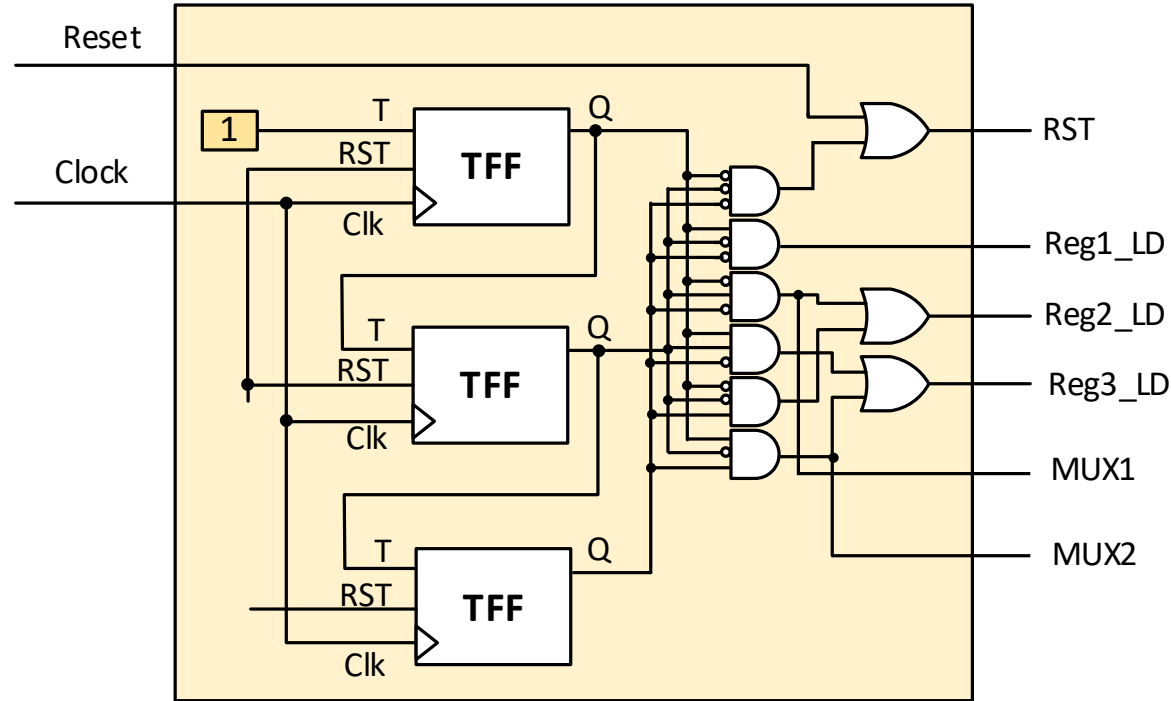
→ Logic design of control signals (based on state transition table)):

- **RST** = Rest + S0
- **reg1_LD** = S1,
- **reg2_LD** = S2 + S4
- **reg3_LD** = S3 + S5
- **MUX1** = S2
- **MUX2** = S5

Current State	State Codes
S0	000
S1	001
S2	010
S3	011
S4	100
S5	101

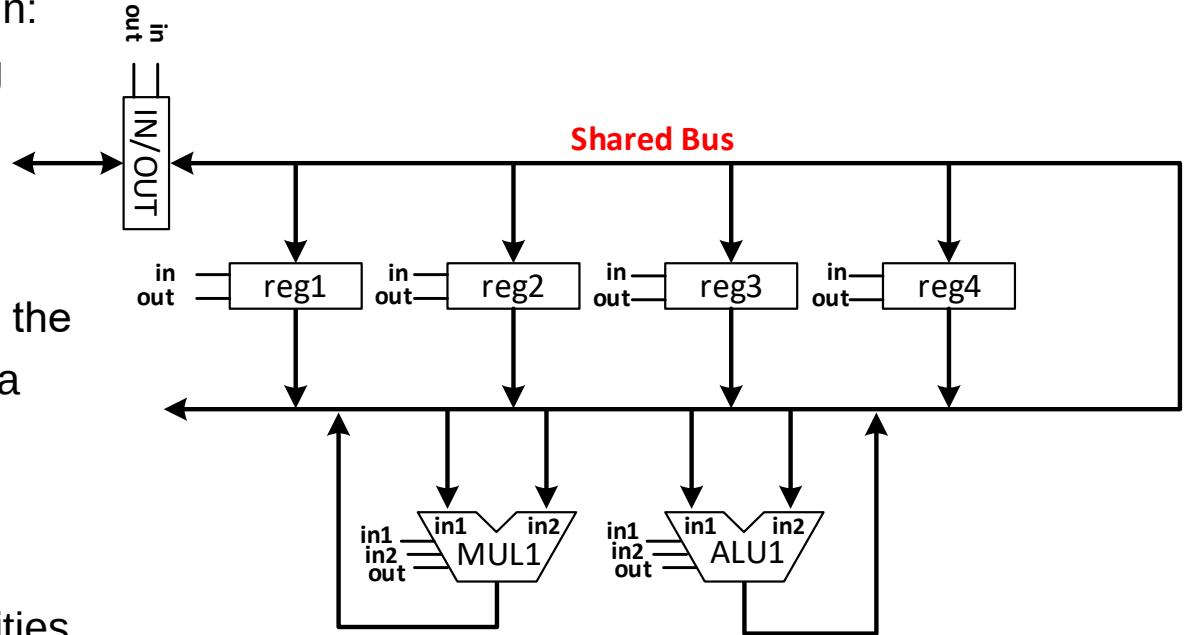
Example 2: A second order polynomial

→ Controller design.



Example 2: A second order polynomial

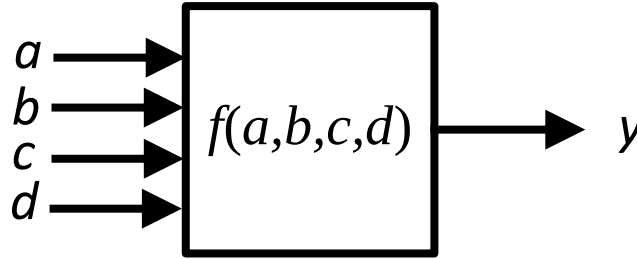
- Datapath design (Note, this is just a design out of so many possibilities)
- Note, if we choose this design:
 - All suggested scheduling can be implemented on this datapath!
 - All input outputs must be tri-state, that's why all the input outputs must have a control signal.
 - Bus becomes a speed bottle-neck!
 - Think about other possibilities.



Example 3:

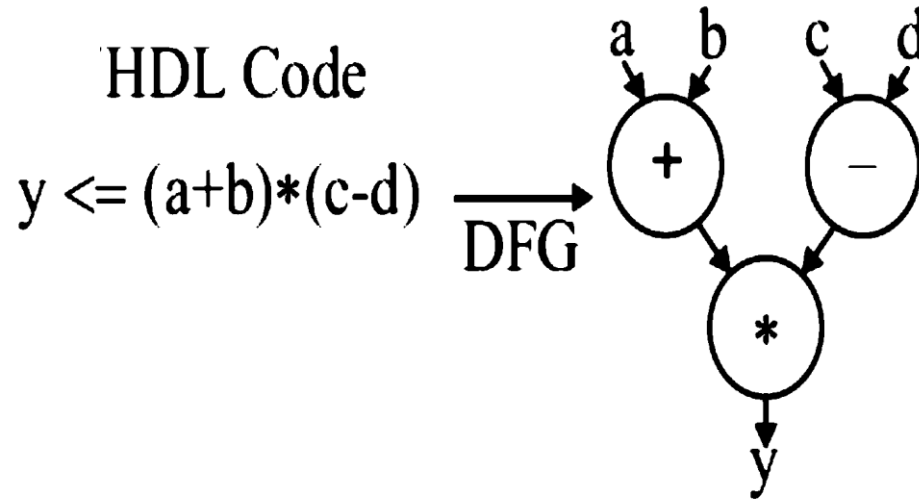
- Let have a look at a small design example to demonstrate the various phases of design steps in detail.
- Suppose that we want to implement a hardware to perform the operation

$$y = (a + b) \times (c - d)$$



Example 3:

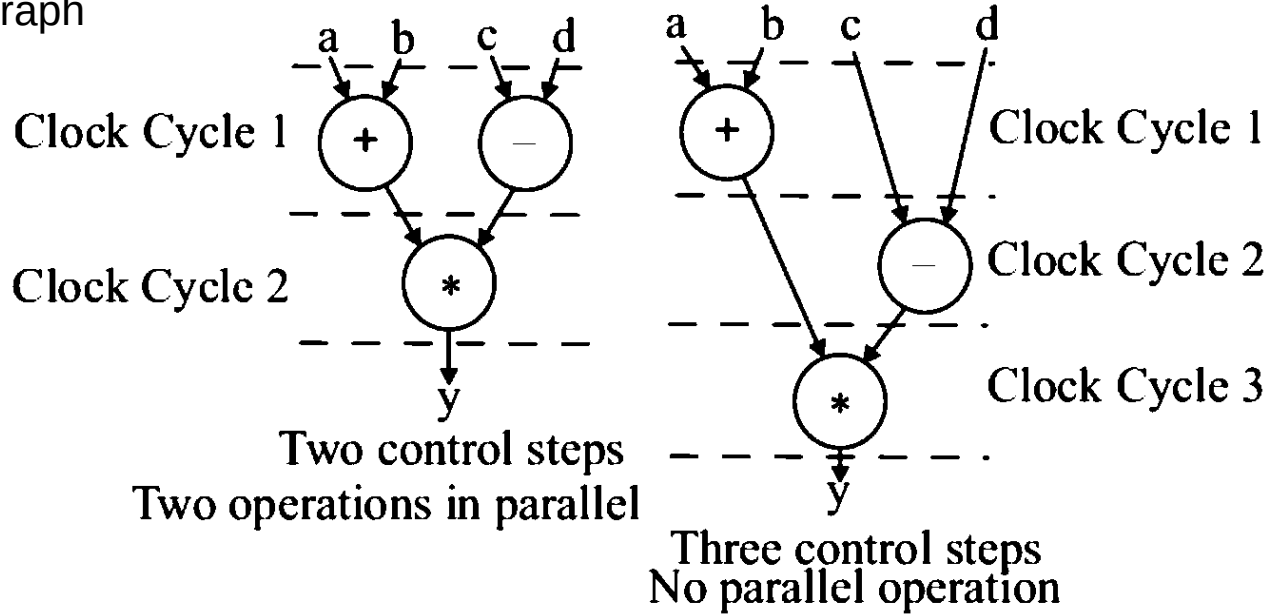
→ Design description



(a) Step 1: Compilation and transformation

Example 3:

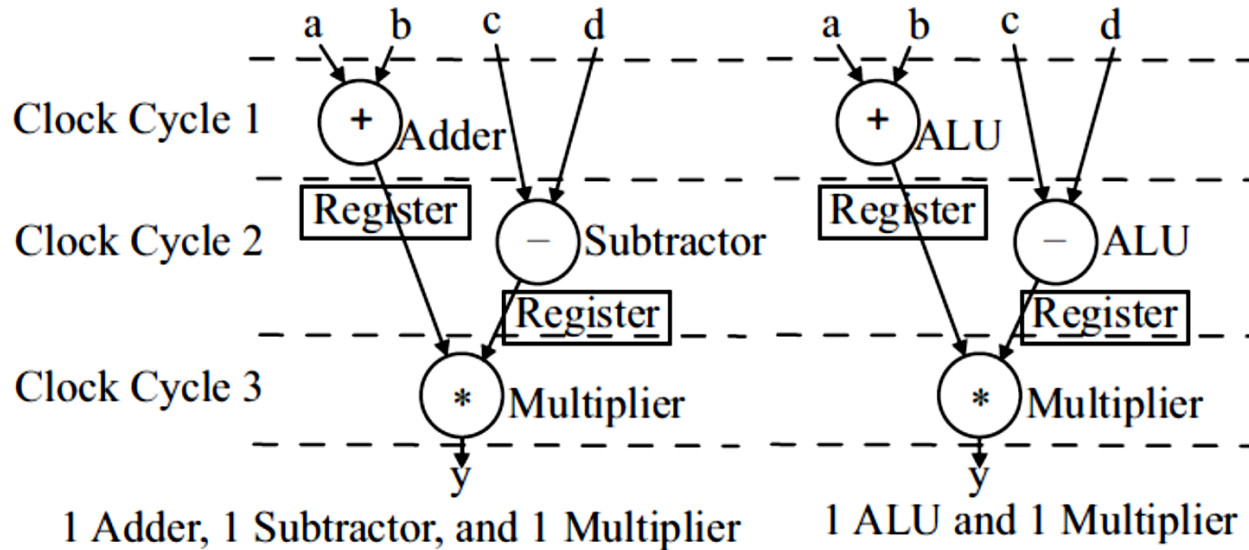
→ Dataflow Graph



(b) Step 2: Scheduling (time or resource constraints)

Example 3:

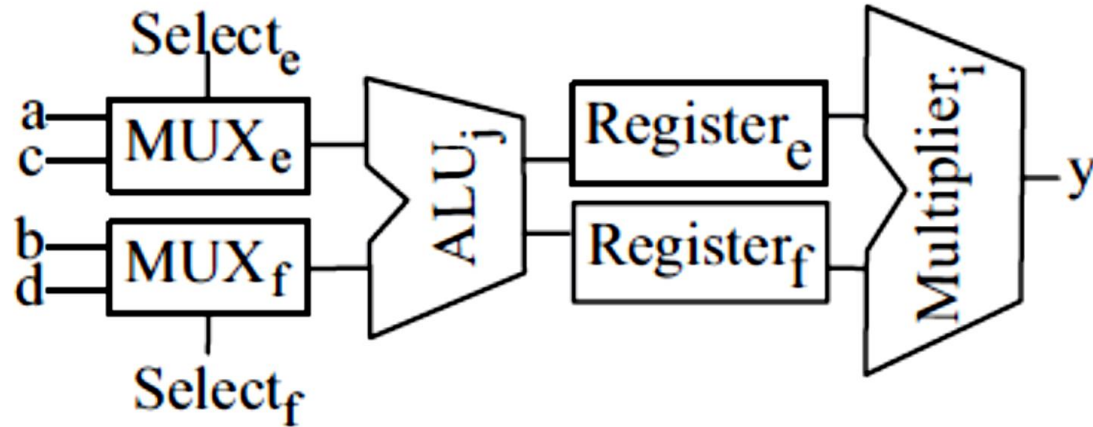
→ Scheduling, Binding, Allocation



(c) Step 3: Allocation (fixed amount and types of resources)

Example 3:

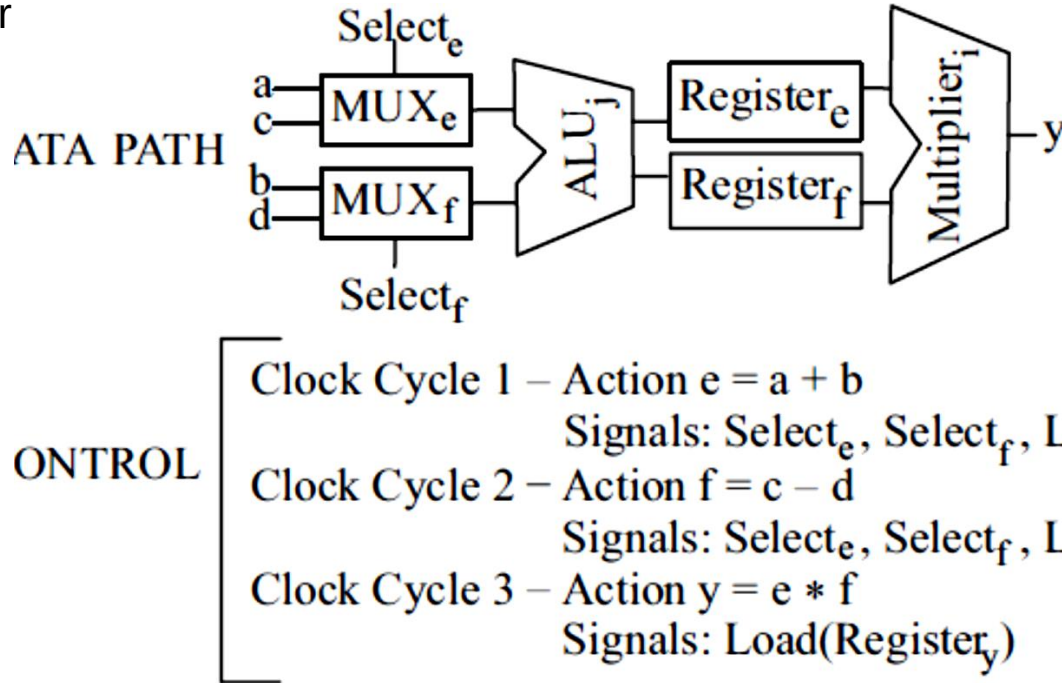
→ Datapath Design



(b) Step 5: Connection allocation (communication between resources: bus, buffer, or MUX)

Example 3:

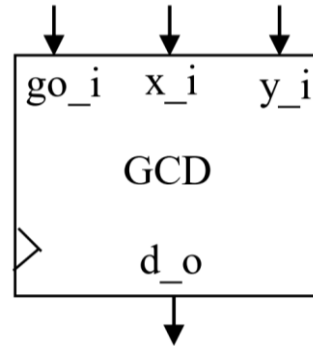
→ Controller



(c) Step 6: Architecture generation (data path and control)

Example 4: greatest common divisor

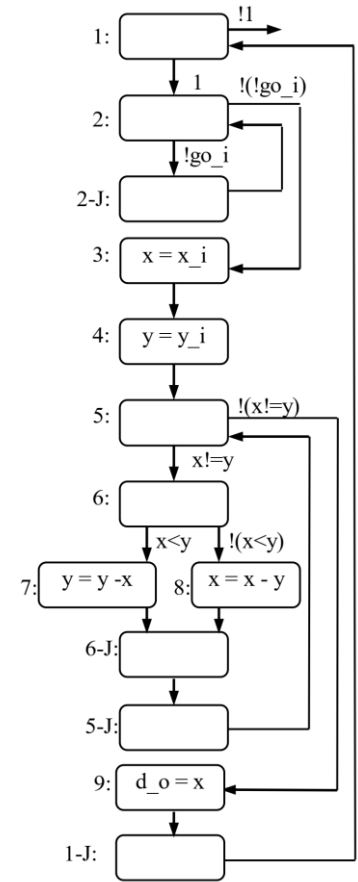
- First create algorithm
- Convert algorithm to “complex” state machine
 - Known as FSM: finite-state machine with datapath
 - Can use templates to perform such conversion



(a) Black-box view

```
0: int x, y;  
1: while (1) {  
2:   while (!go_i);  
3:   x = x_i;  
4:   y = y_i;  
5:   while (x != y) {  
6:     if (x < y)  
7:       y = y - x;  
8:     else  
9:       x = x - y;  
9:   }  
9:   d_o = x;  
}
```

(b) Pseudocode of the design functionality

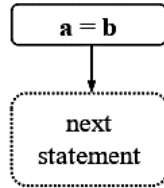


(c) State diagram

State diagram templates

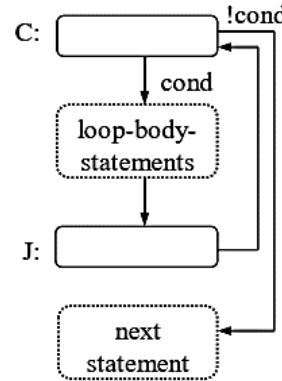
Assignment statement

```
a = b
next statement
```



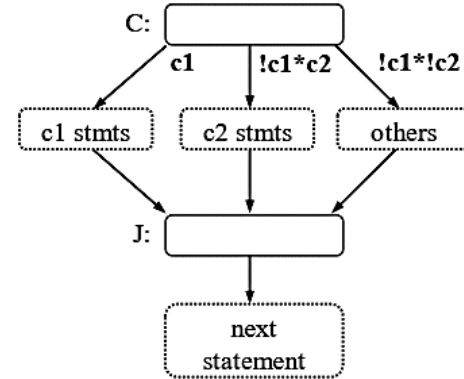
Loop statement

```
while (cond) {
    loop-body-
    statements
}
next statement
```



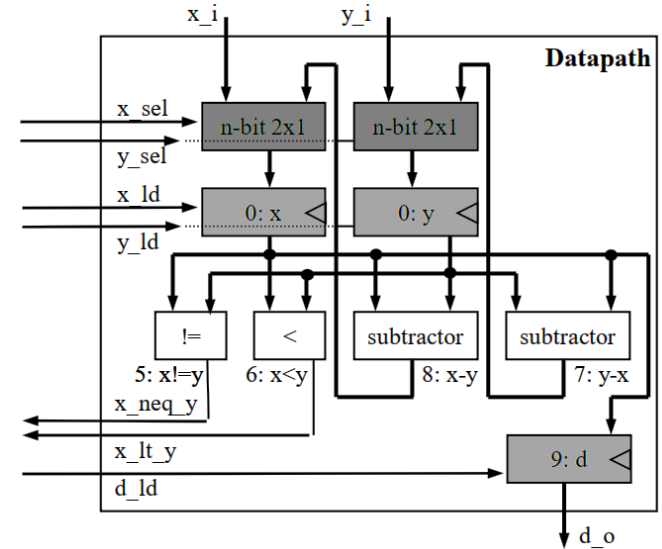
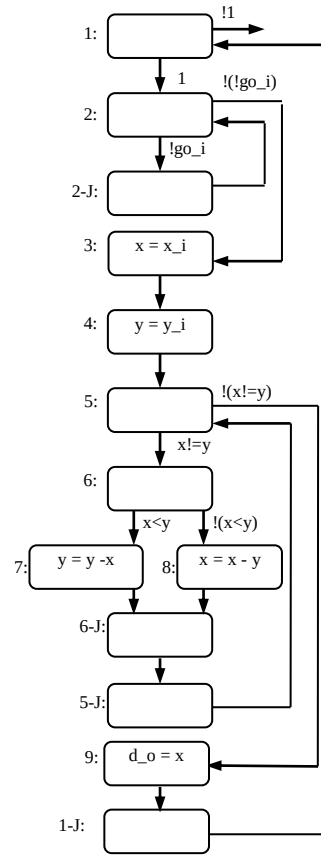
Branch statement

```
if (c1)
    c1 stmts
else if c2
    c2 stmts
else
    other stmts
next statement
```

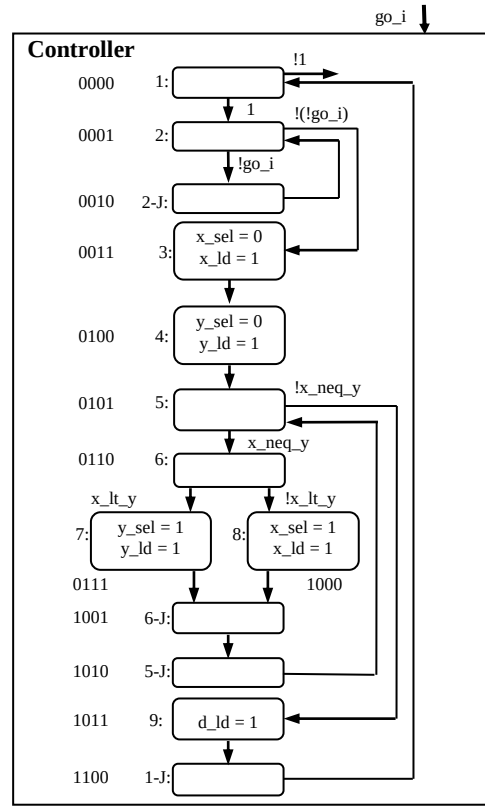
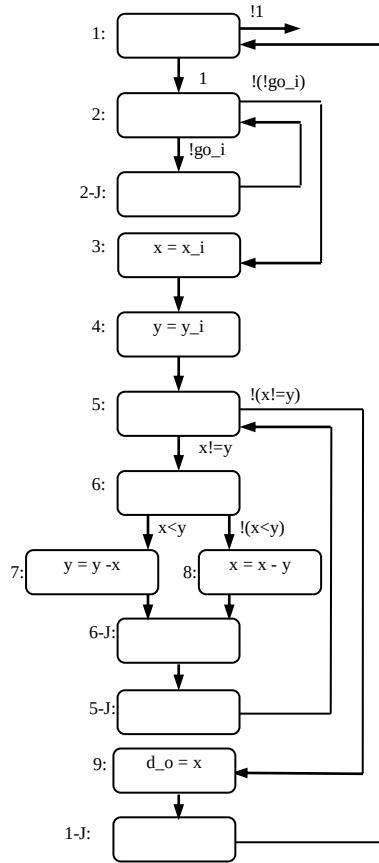


Creating the datapath

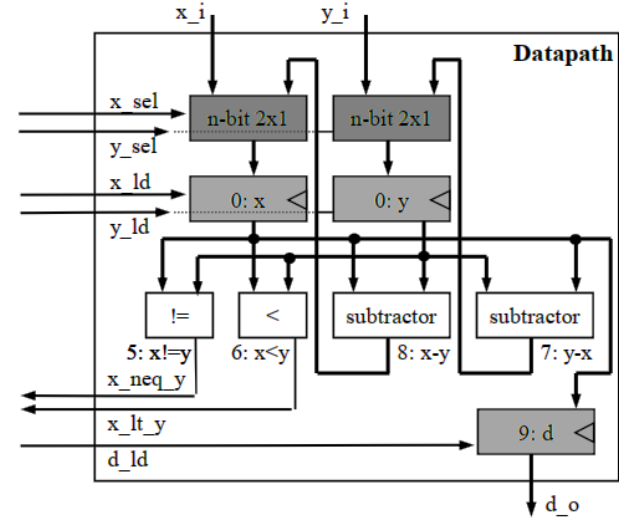
- Create a register for any declared variable
- Create a functional unit for each arithmetic operation
 - Based on reads and writes
 - Use multiplexors for multiple sources
- Create unique identifier
 - for each datapath component control input and output



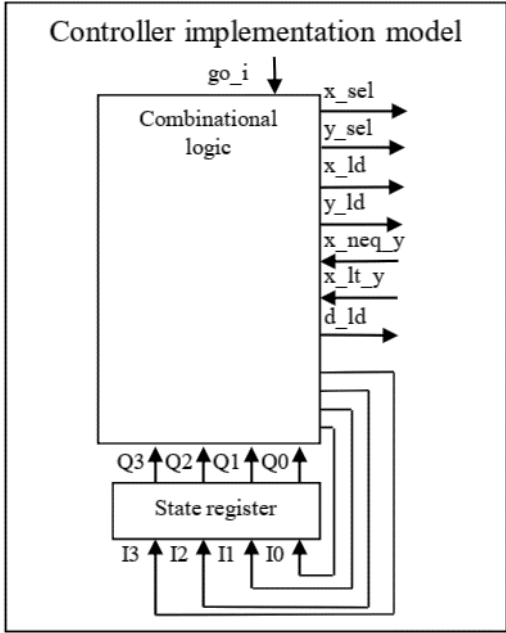
Creating the controller's FSM



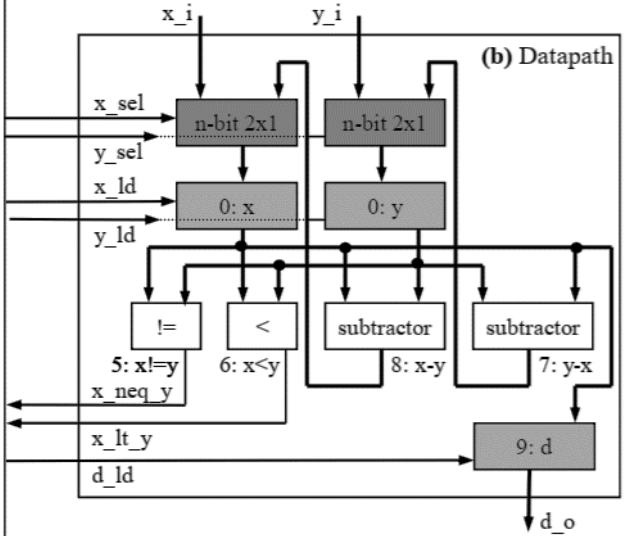
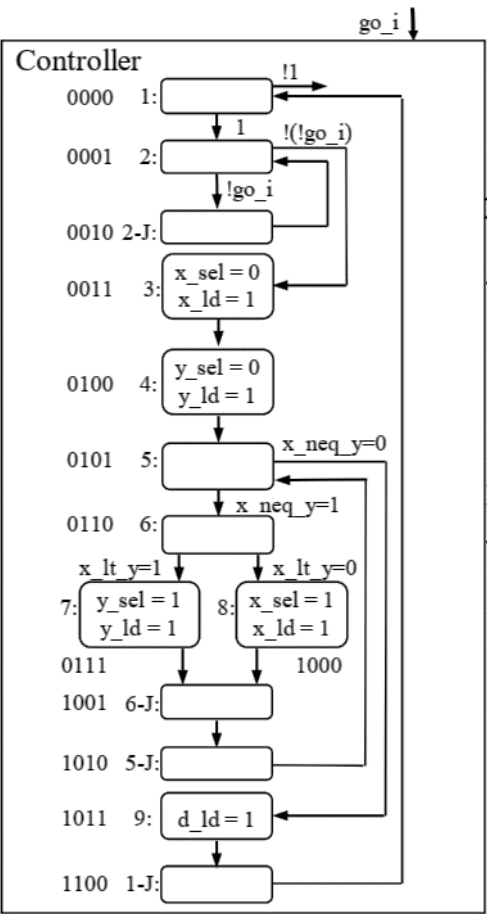
- Same structure as FSMD
- Replace complex actions/conditions with datapath configurations



Splitting into a controller and datapath



--->

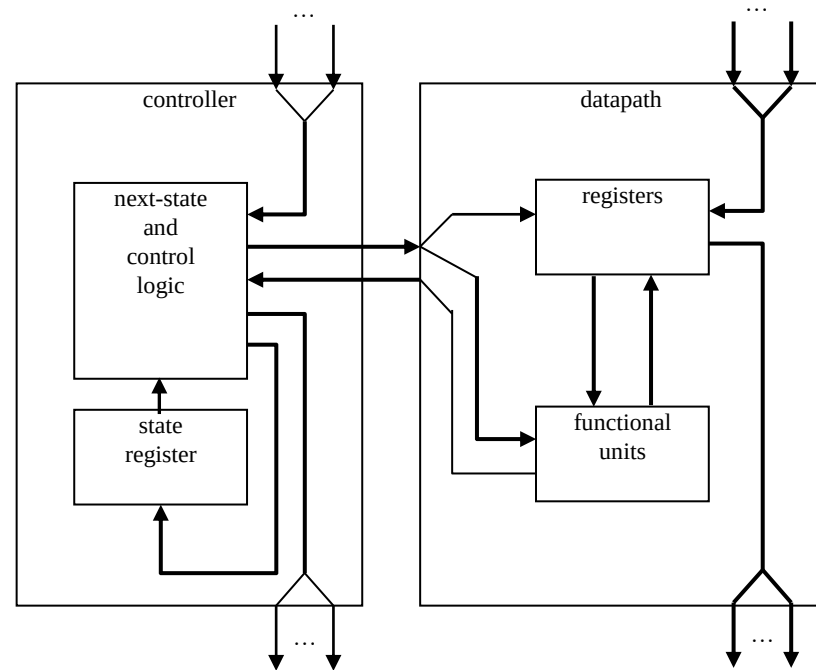


Controller state table for the GCD example

Inputs							Outputs								
Q3	Q2	Q1	Q0	x_neq_y	x_lt_y	go_i	I3	I2	I1	I0	x_sel	y_sel	x_ld	y_ld	d_ld
0	0	0	0	*	*	*	0	0	0	1	X	X	0	0	0
0	0	0	1	*	*	0	0	0	1	0	X	X	0	0	0
0	0	0	1	*	*	1	0	0	1	1	X	X	0	0	0
0	0	1	0	*	*	*	0	0	0	1	X	X	0	0	0
0	0	1	1	*	*	*	0	1	0	0	0	X	1	0	0
0	1	0	0	*	*	*	0	1	0	1	X	0	0	1	0
0	1	0	1	0	*	*	1	0	1	1	X	X	0	0	0
0	1	0	1	1	*	*	0	1	1	0	X	X	0	0	0
0	1	1	0	*	0	*	1	0	0	0	X	X	0	0	0
0	1	1	0	*	1	*	0	1	1	1	X	X	0	0	0
0	1	1	1	*	*	*	1	0	0	1	X	1	0	1	0
1	0	0	0	*	*	*	1	0	0	1	1	X	1	0	0
1	0	0	1	*	*	*	1	0	1	0	X	X	0	0	0
1	0	1	0	*	*	*	0	1	0	1	X	X	0	0	0
1	0	1	1	*	*	*	1	1	0	0	X	X	0	0	1
1	1	0	0	*	*	*	0	0	0	0	X	X	0	0	0
1	1	0	1	*	*	*	0	0	0	0	X	X	0	0	0
1	1	1	0	*	*	*	0	0	0	0	X	X	0	0	0
1	1	1	1	*	*	*	0	0	0	0	X	X	0	0	0
1	1	1	1	*	*	*	0	0	0	0	X	X	0	0	0

Completing the GCD custom single-purpose processor design

- We finished the datapath
- We have a state table for the next state and control logic
 - All that's left is combinational logic design
- This is *not* an optimized design, but we see the basic steps



a view inside the controller and datapath

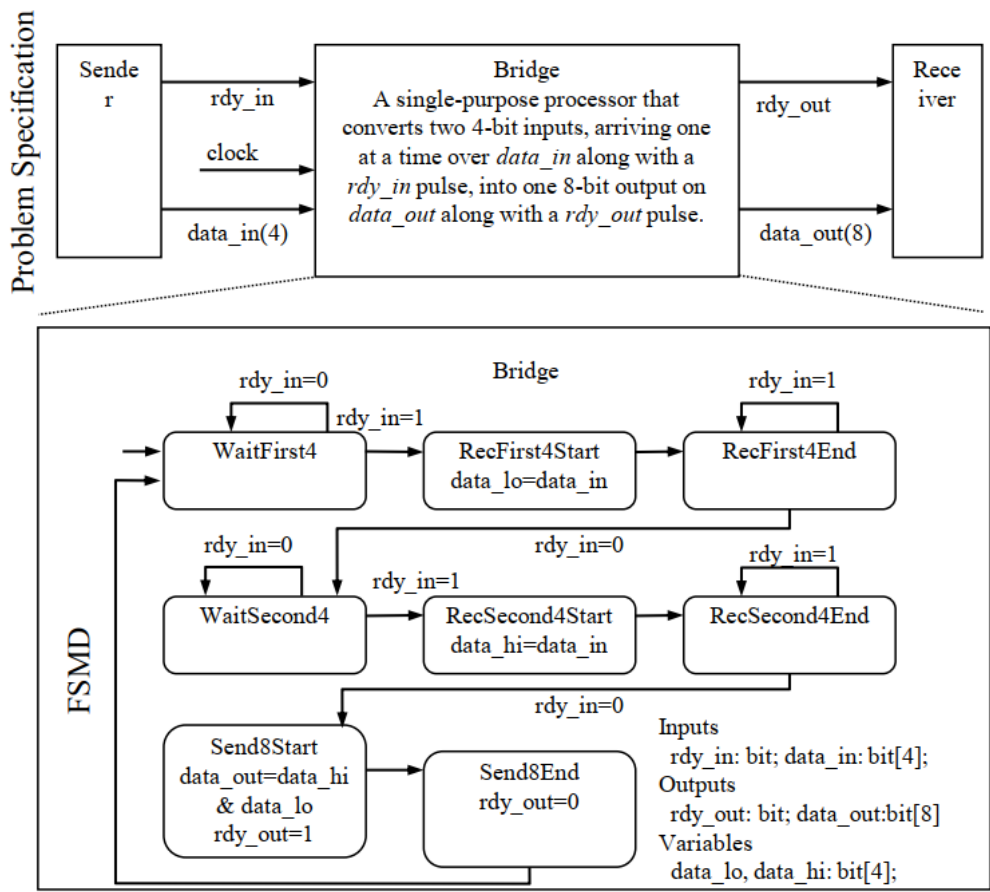
More Design Challenges

Design Optimization

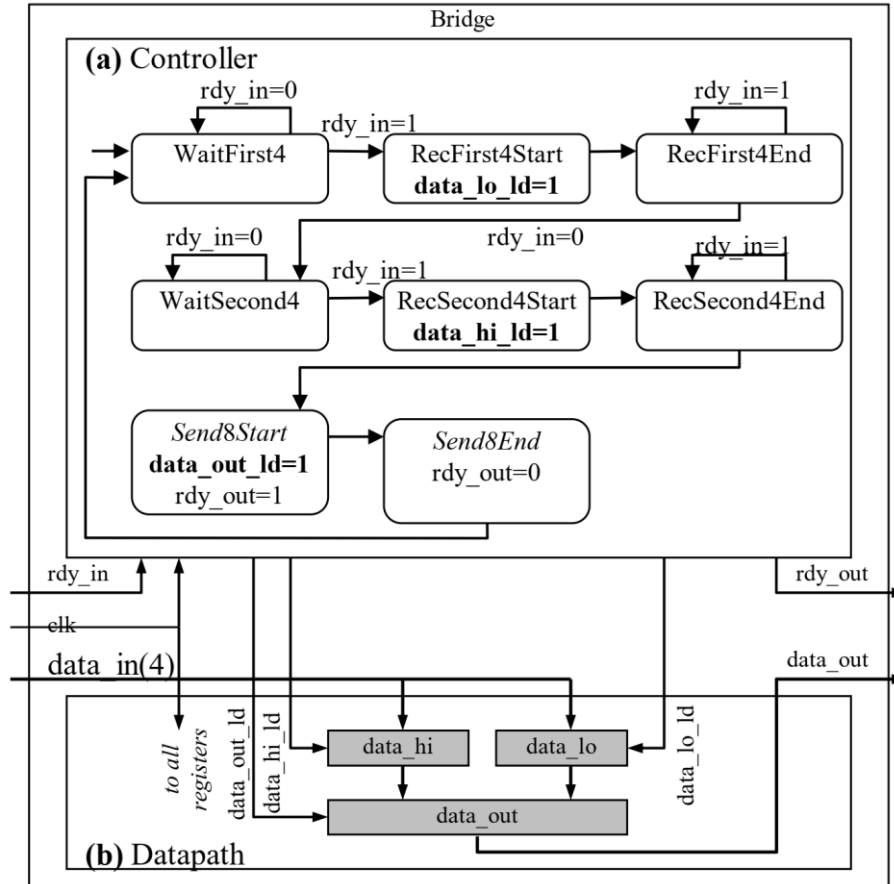
Optimisation de la conception

RT-level custom single-purpose processor design

- We often start with a state machine
 - Rather than algorithm
 - Cycle timing often too central to functionality
- Example
 - Bus bridge that converts 4-bit bus to 8-bit bus
 - Start with FSMD
 - Known as register-transfer (RT) level
 - Exercise: complete the design



RT-level custom single-purpose processor design (cont')



Optimizing the original program

- Optimization is the task of making design metric values the best possible
- Optimization opportunities
 - original program
 - FSMD
 - datapath
 - FSM
- Analyze program attributes and look for areas of possible improvement
 - number of computations
 - size of variable
 - time and space complexity
 - operations used
 - multiplication and division very expensive

Optimizing the original program (cont')

original program

```
0: int x, y;
1: while (1) {
2:   while (!go_i);
3:   x = x_i;
4:   y = y_i;
5:   while (x != y) {
6:     if (x < y)
7:       y = y - x;
8:     else
9:       x = x - y;
10:  }
11:  d_o = x;
12: }
```

replace the subtraction
operation(s) with modulo
operation in order to speed
up program

optimized program

```
0: int x, y, r;
1: while (1) {
2:   while (!go_i);
3:   // x must be the larger number
4:   if (x_i >= y_i) {
5:     x=x_i;
6:     y=y_i;
7:   }
8:   else {
9:     x=y_i;
10:    y=x_i;
11:  }
12:  while (y != 0) {
13:    r = x % y;
14:    x = y;
15:    y = r;
16:  }
17:  d_o = x;
18: }
```

GCD(42, 8) - 9 iterations to complete the loop

x and y values evaluated as follows : (42, 8), (43, 8),
(26,8), (18,8), (10, 8), (2,8), (2,6), (2,4), (2,2).

GCD(42,8) - 3 iterations to complete the loop

x and y values evaluated as follows: (42, 8), (8,2),
(2,0)

Optimizing the FSMD

→ Areas of possible improvements

➤ Merge states

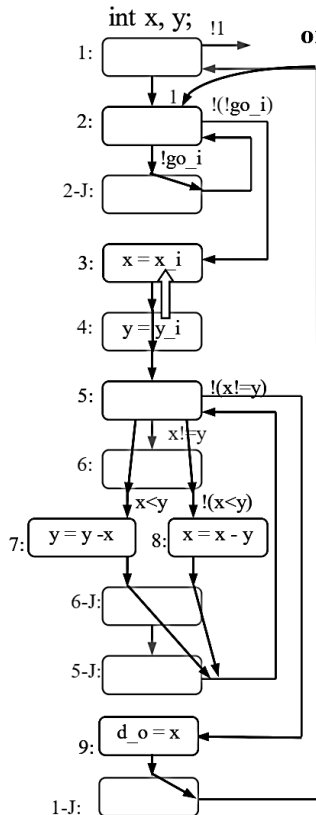
- states with constants on transitions can be eliminated, transition taken is already known
- states with independent operations can be merged

➤ Separate states

- states which require complex operations ($a*b*c*d$) can be broken into smaller states to reduce hardware size

➤ Scheduling

Optimizing the FSMD (cont.)



eliminate state 1 – transitions have constant values

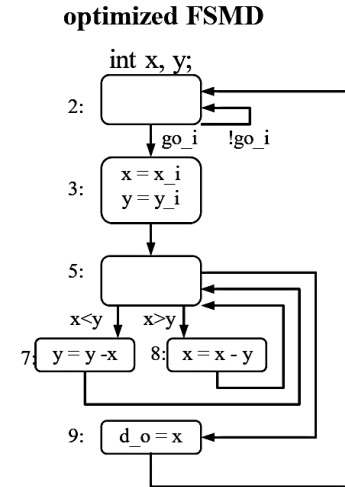
merge state 2 and state 2J – no loop operation in between them

merge state 3 and state 4 – assignment operations are independent of one another

merge state 5 and state 6 – transitions from state 6 can be done in state 5

eliminate state 5J and 6J – transitions from each state can be done from state 7 and state 8, respectively

eliminate state 1-J – transition from state 1-J can be done directly from state 9



Optimizing the datapath

- Sharing of functional units
 - one-to-one mapping, as done previously, is not necessary
 - if same operation occurs in different states, they can share a single functional unit
- Multi-functional units
 - ALUs support a variety of operations, it can be shared among operations occurring in different states

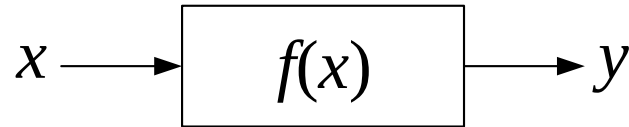
Optimizing the FSM

- State encoding
 - task of assigning a unique bit pattern to each state in an FSM
 - size of state register and combinational logic vary
 - can be treated as an ordering problem
- State minimization
 - task of merging equivalent states into a single state
 - state equivalent if for all possible input combinations the two states generate the same outputs and transitions to the next same state

Example: an ODE

A system which is represented by a differential equation.

$$\frac{d^2y}{dx^2} + 3 \frac{dy}{dx} x + 3y = 0$$



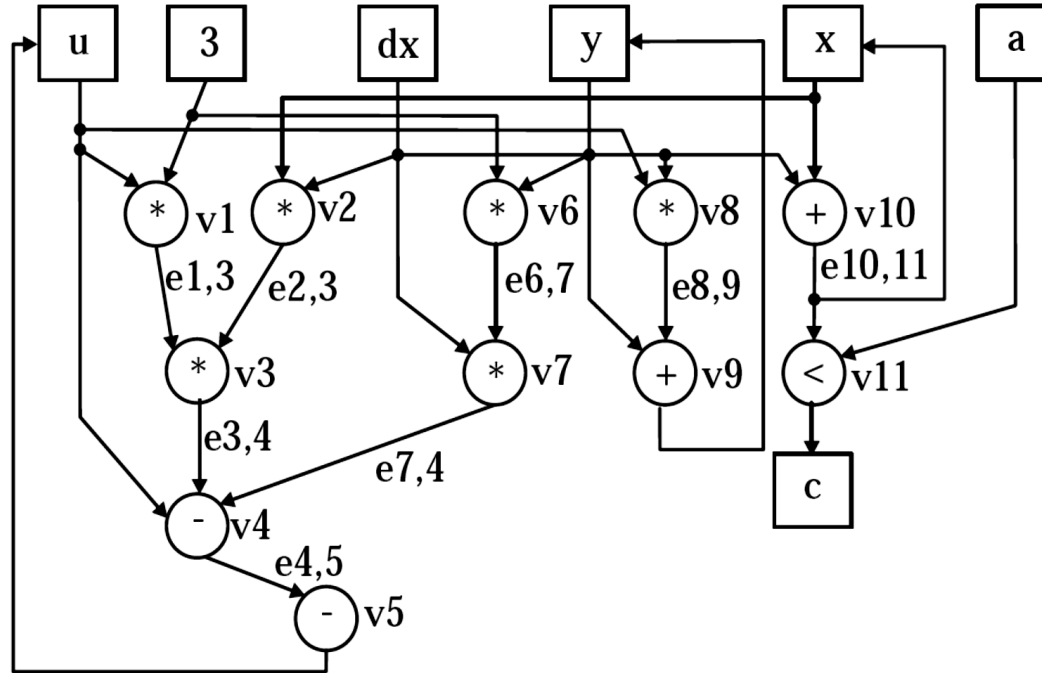
Example: an ODE

A pseudo-code description of the equation:

```
while (x < a) do
  x1 := x + dx;
  u1 := u - ( 3 * x * u * dx) - (3 * y * dx);
  y1 := y + ( u * dx);
  c = x < a;
  x := x1;
  u := u1;
  y := y1;
end while
```

Example: an ODE

→ Dataflow Graph



Example: an ODE

→ Class notes:

Summary

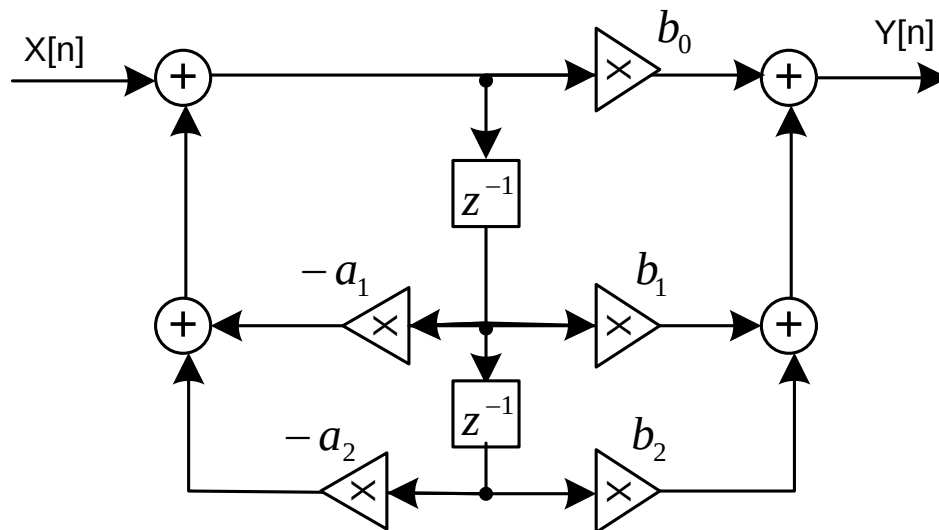
- Custom single-purpose processors
 - Straightforward design techniques
 - Can be built to execute algorithms
 - Typically start with FSMD
 - CAD tools can be of great assistance

Appendix A

More Desing Examples

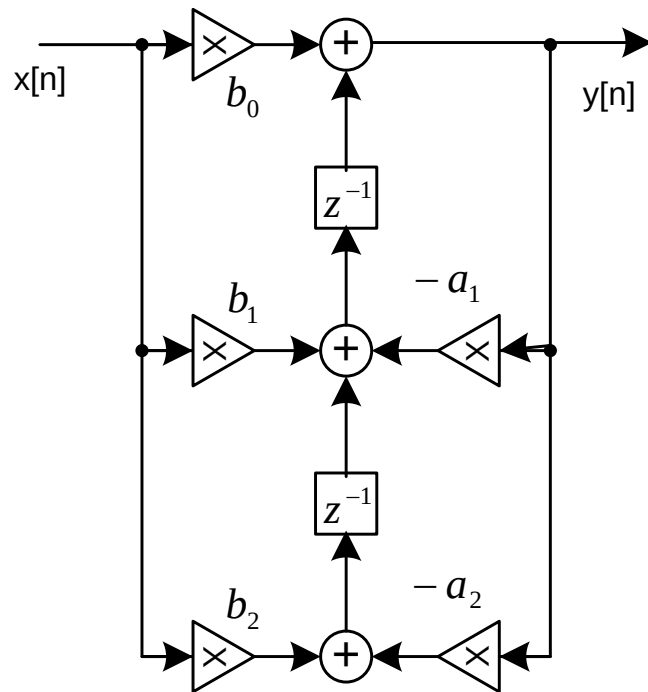
Supplementary Examples (1)

- Implementation of general tow-pole IIR filter
- simple form: dataflow graph



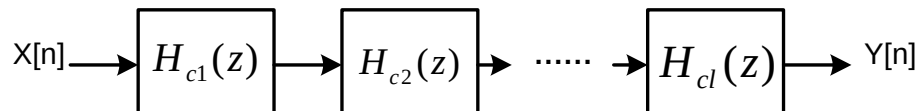
Supplementary Examples (2)

- Implementation of general tow-pole IIR filter
- transposed form: dataflow graph

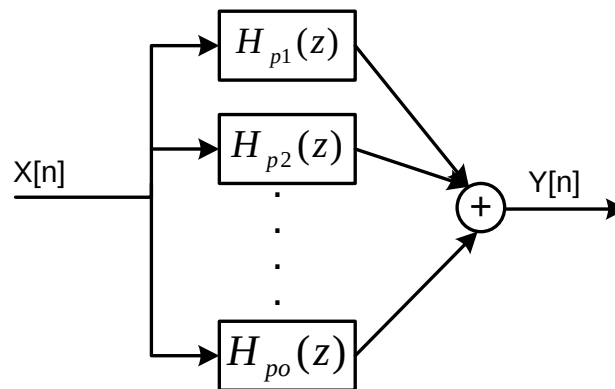


Supplementary Examples (3)

- A higher order filter
- It can be decomposed in
 - a) cascade form,
 - b) parallel form.



(a)

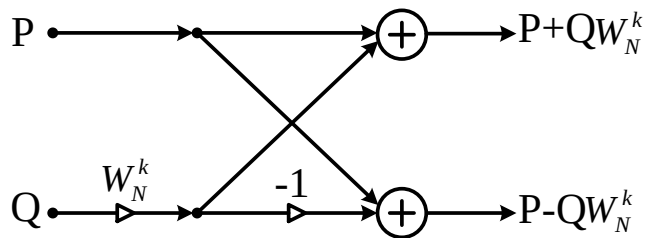


(b)

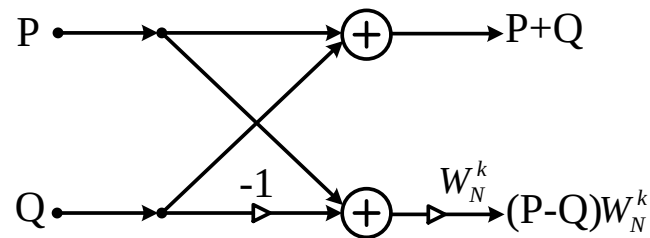
Supplementary Examples (4)

→ Raxid-2 Butterfly Cell for FFT

- a) DIT Implementation
- b) DIF Implementation.



(a)



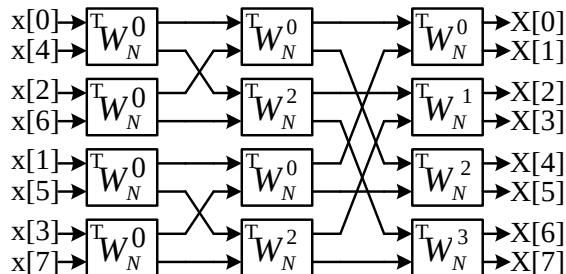
(b)

Supplementary Examples (5)

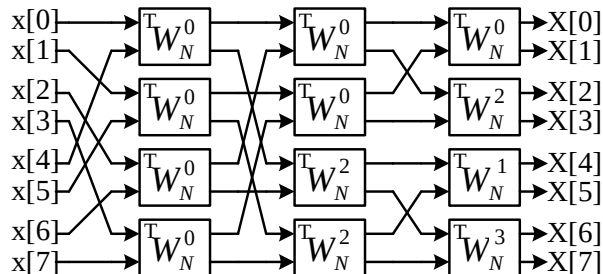
- 8-point implementation of FFT by radix-2 Butterfly cells (each block is a radix-2 butterfly cell)
- a) DIT bit reversed ordered inputs
 - b) DIT normal ordered inputs
 - c) DIF bit-reversed ordered inputs
 - d) DIF normal ordered inputs

Supplementary Examples (5)

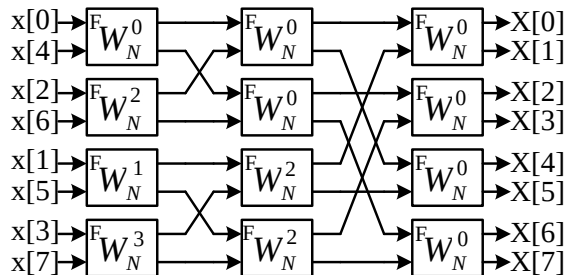
→ 8-point implementation of FFT by radix-2 Butterfly cells (cont')



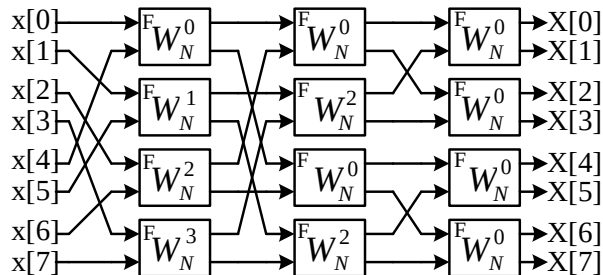
(a)



(b)



(c)



(d)